

User Manual to integrate the functionalities of FGI-OSNMA within FGI-GSRx

This user manual is intended to help create the Python environment to run FGI-OSNMA, along with the requirements and steps required to interface the FGI-GSRx with FGI-OSNMA. It is important to note that OSNMA support is only available for Galileo E1 signal.

(1) Create a Python environment or use an already existing one.

I. For an already existing environment, activate it:

- *conda activate FGlosnmaenv*

II. Or create a new environment by using the following command and activate it.

- *conda create -n FGlosnmaenv Python=3.10 # use Python version that is installed or is intended to be used.*
- *conda activate FGlosnmaenv*

(2) Install the required packages for FGI-OSNMA if not already installed in the environment. The following are the required packages and versions. It would be a good practice to run FGI-OSNMA in Python first to ensure that the block is running.

(Note: The requirements in future can be changed. Please consider checking the requirements directly from the FGI-OSNMA GitHub.

Function	Version
bitstring	3.1.9
crccheck	1.3.0
cryptography	41.0.5
leapseconds	2023.4.0
numpy	1.25.1
pyserial	3.5
matplotlib	3.7.2

(3) Get the MATLAB Python interface ready by doing the following steps:

- I. In the MATLAB command prompt, run the following command. Ensure that the same Python environment is activated as mentioned in step 2.
- II. *pyenv('Version', 'C:\Program Files\Miniconda3\envs\FGlosnmaenv\Python.exe', 'ExecutionMode', 'OutOfProcess')*
- III. If you receive the following output, then Python is activated.

PythonEnvironment with properties:

Version: "3.8"

Executable: "/home/username/py38/bin/Python"

Library: "<library path>/libPython3.8.so"

Home: "/home/username/py38"

Status: NotLoaded

ExecutionMode: OutOfProcess

- (4) The public key needed to initiate FGI-OSNMA is to be placed under the folder 'osnma'. The public key file (e.g., "osnma-key-aug.pem") is to be provided by the user. In the param file, the following parameter needs to be defined:

```
osnma,publicKeyName,"-k .\\osnma\\osnma-key-aug.pem",
```

Important: The file name should always be under the folder 'osnma'. It is recommended to only change the public key file name without changing the path.

- (5) The status of TESLA root key file must be defined in the following manner.

- I. It can be retrieved from the data only if the data length is sufficient for the retrieval. In this case following option should be used in the param file:

```
osnma,externalTESLARootKey,false,
```

- II. The user can also provide an external TESLA ROOT key (e.g., 'kroot_1257_80'). This file should be placed similarly under the folder 'osnma'. In this case, the following setting is to be enabled in the param file:

```
osnma,externalTESLARootKey,true,
```

```
osnma,externalTESLARootkeyFilename,"-r .\\osnma\\kroot_1257_80",
```

Important: The file name should always be under the folder 'osnma'. It is recommended to only change the tesla root key file name without changing the path.

- (6) You can check at this point whether the interface is successful by calling the following function from the MATLAB prompt:

- I. `pyrunfile(".\\osnma\\app\\osnma-cli.py -i galrawinav-270923.sbf -k .\\osnma\\osnma-key-aug.pem -p sbf"); % for -sbf input.`

- II. `pyrunfile(".\\osnma\\app\\osnma-cli.py -i .\\osnma\\OSNMA_HEX_STR.txt -p ascii -k .\\osnma\\osnma-key-aug.pem -r .\\osnma\\kroot_1257_80"); % for -ascii input.`

- (7) After establishing and verifying the MATLAB and Python interface, do the following steps:

- I. Define the following settings in the param file:

– **% System parameters**

```
sys,enabledSignals,[["gale1b"]], % List of enabled signals
```

The receiver currently supports OSNMA-based authentication with Galileo E1B signal.

– **%OSNMA parameters**

```
osnma,enableOSNMA detection,true, % OSNMA_flag (false = Disable or true = enable OSNMA detection)
```

```
osnma,publicKeyName,"-k .\\osnma\\osnma-key-aug.pem", %File containing public key
```

```
osnma,externalTESLARootKey,true, %If external root key is to be used, set this parameter to true
```

```
osnma,externalTESLARootkeyFilename,"-r .\\osnma\\kroot_1257_80", % File containing external TESLA root key
```

- II. Run the FGI-GSRx main function by using the following command:

```
gsrx("paramfilename");
```

Note:

- A good helping guide to interface Python with MATLAB can be found on the following link:
<https://se.mathworks.com/matlabcentral/answers/1750425-Python-virtual-environments-with-matlab>
- The ASCII input file "OSNMA_HEX_STR.txt" generated by FGI-GSRx is fed to FGI-OSNMA for OSNMA-related processing.
- Both the input ASCII file and the output of FGI-OSNMA are also saved under the 'osnma' folder by default.
- The FGI-GSRx-v2.1.0 architecture is presented in Figure 1. A detail explanation of the architecture with a couple of use cases is presented in [1]. FGI-GSRx users are encouraged to refer to the following publication when utilizing the receiver for OSNMA-based position authentication:

[1] Liaquat, M.; Bhuiyan, M.Z.H.; Hammarberg, T.; Islam, S.; Saajasto, M.; Kaasalainen, S. An End-To-End Solution Towards Authenticated Positioning Utilizing Open-Source FGI-GSRx and FGI-OSNMA. *Eng. Proc.* **2025**, *88*, 58.
<https://doi.org/10.3390/engproc2025088058>.

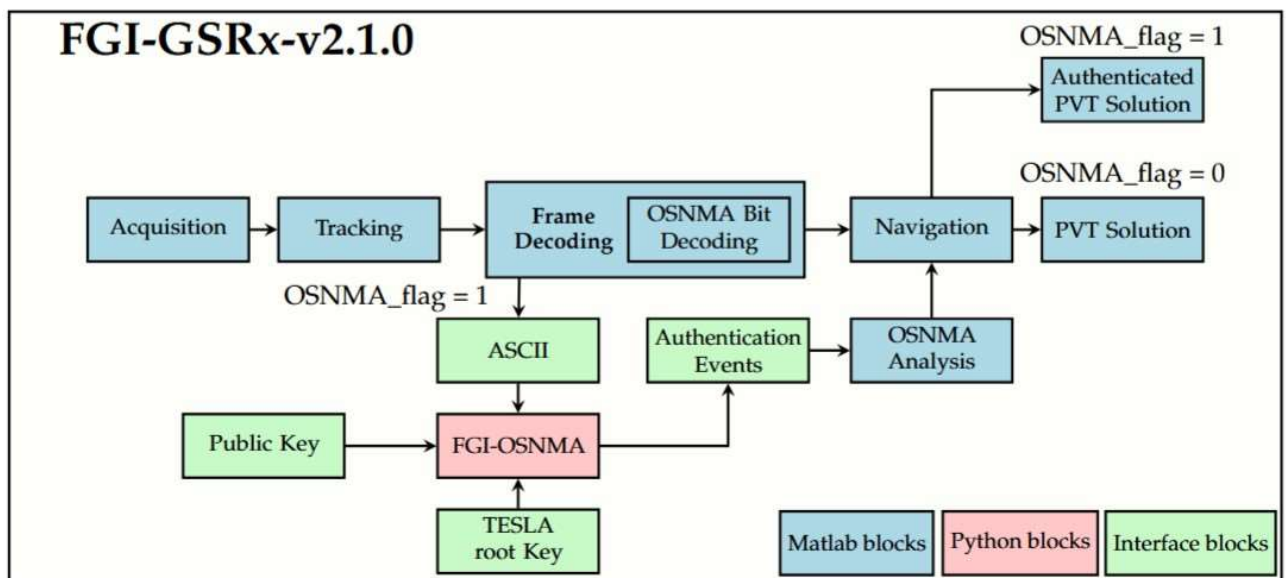


Figure 1: FGI-GSRx-v2.1.0 architecture [1]